



A Course End Project Report on
Restaurant Order Management System
A Full-Stack Web Application with a PostgreSQL Database

A9511 - Database Management Systems

Submitted by
Batch No. 6

Madhavarapu Saritha	25881A05V7
Chepyala Vishal	25881A05X9
Gundu Srijay Krishna	25881A05X0

Course Facilitator
Dr. Reddy Saisindhutheja
Associate Professor

Department of Computer Science and Engineering

Vardhaman College of Engineering, Hyderabad

November 2026

Contents

1	Problem Definition and Context	2
1.1	Restaurant Operations as a Database Problem	2
1.2	Relevance to Sustainable Development Goals (SDGs)	2
2	Problem Analysis and Solution Development	3
2.1	Characterization of the Problem	3
2.2	Engineering Knowledge Applied	3
2.3	System Architecture	3
2.4	Database Design	4
2.5	Business Rules Inside the Database	5
2.6	Analytical Queries for the Dashboard	8
2.7	Application Layer	9
2.8	User Interface	10
2.9	Deployment	16
2.10	Testing and Validation	16
2.11	Proposed Solution Strategy	17
3	Sustainability Integration	17
3.1	Reducing Food Waste and Paper	17
3.2	Impact on Society and Environment	17
4	Conclusion and Future Directions	17
4.1	Solution Summary	17
4.2	Future Scope	18

1 Problem Definition and Context

1.1 Restaurant Operations as a Database Problem

A busy restaurant runs several processes in parallel. Waiters take orders at many tables, the kitchen cooks dishes in a sensible sequence, the cashier prepares GST bills and collects payments, and the manager watches sales and stock. In many small restaurants these processes still run on handwritten kitchen order tickets (KOTs) and a billing calculator. As a result, orders are lost or misread, dishes reach the wrong table, bills are calculated wrongly, tables wait for a bill, and nobody knows that an ingredient has run out until a customer orders the dish.

Problem Statement: Design and build a *live*, full-stack Restaurant Order Management System (RMS) backed by a relational database. It must manage the menu, dining tables, orders from placement to payment, a kitchen display, GST billing, ingredient inventory, and management reports, and it must stay consistent when many staff members use it at the same time.

Objectives:

- Design a normalized PostgreSQL schema for menu, tables, staff, orders, order items, bills, payments, ingredients and recipes.
- Enforce business rules *inside the database*: order and dish status transitions, automatic stock deduction, one open order per table, and GST bill calculation.
- Build a REST API (Node.js/Express) with secure login and role-based access for Manager, Waiter, Chef and Cashier.
- Build a responsive web frontend: floor plan, order taking, kitchen display, billing counter, menu, inventory and an analytics dashboard.
- Deploy the system live on the Internet and verify it with functional, rule and role tests.

Live application: <https://spice-route-rms.onrender.com>

Source code: <https://github.com/logicnest-alpha/contest-repo-01> (folder `Course_End_Project_Restaurant_OMS`)

Demo logins: manager / manager123, waiter / waiter123, chef / chef123, cashier / cashier123

1.2 Relevance to Sustainable Development Goals (SDGs)

- **SDG 8 (Decent Work and Economic Growth):** Affordable digital tools raise the productivity of small and medium restaurants (Target 8.2) and reduce billing errors and revenue leakage.
- **SDG 12 (Responsible Consumption and Production):** Recipe-based stock tracking and sales analytics help a kitchen prepare and buy only what is needed, reducing food waste (Target 12.3).
- **SDG 9 (Industry, Innovation and Infrastructure):** A cloud-hosted, browser-based system gives small enterprises access to information technology without expensive hardware (Target 9.3).

2 Problem Analysis and Solution Development

2.1 Characterization of the Problem

The RMS is a multi-user, transaction-oriented system. Its main difficulties are:

- **Concurrency:** two waiters may try to seat guests at the same table, and two orders may compete for the last kilogram of mutton.
- **State consistency:** a table is “free”, “occupied”, “food ready” or “waiting for bill” depending on its open order, and an order’s status depends on the status of each of its dishes. These must never disagree.
- **Financial correctness:** the bill must use the price at the time of ordering, apply discounts and GST correctly, and close only when fully paid.
- **Different users, different rights:** a chef must not take payments and a cashier must not see management reports.

Placing these rules in the database (constraints, triggers and stored functions) rather than only in the web code means that *every* program that touches the data obeys them.

2.2 Engineering Knowledge Applied

- **Database Design:** ER modelling, normalization (3NF/BCNF), keys, `CHECK` constraints, partial unique indexes and views.
- **Procedural SQL:** PL/pgSQL functions and `BEFORE/AFTER` row triggers implementing a state machine and inventory updates.
- **Transactions and Concurrency Control:** atomic functions, row locks, advisory locks and constraint-based guarantees.
- **Web Engineering:** a three-tier architecture, REST API design, JSON, single-page application with hash routing.
- **Security:** bcrypt password hashing, JSON Web Tokens (JWT), role-based authorization, parameterized SQL, a least-privilege database role and HTTP security headers.
- **Cloud Deployment:** Platform-as-a-Service hosting (Render), managed PostgreSQL (Supabase) and connection pooling.

2.3 System Architecture

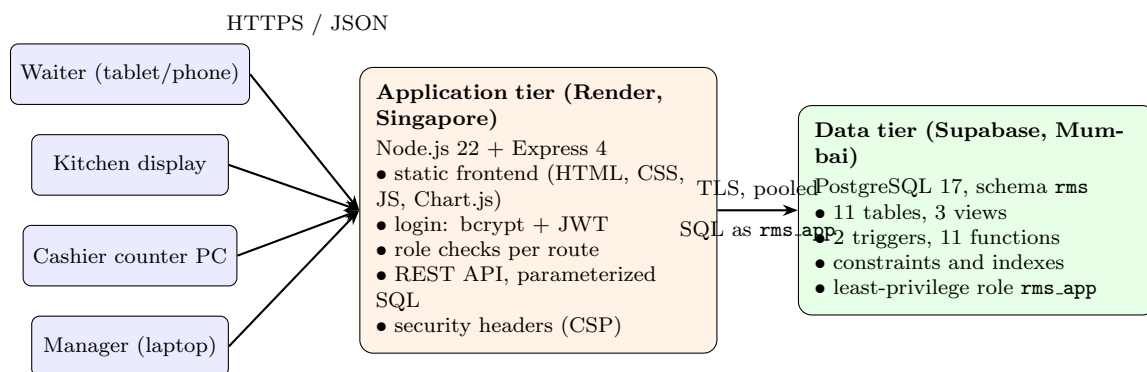


Figure 1: Three-tier architecture of the deployed system

Table 1: Technology stack

Layer	Technology	Purpose
Database	PostgreSQL 17 (Supabase)	Tables, constraints, triggers, PL/pgSQL functions, views (895 lines of SQL)
Backend	Node.js 22, Express 4, <code>pg</code> , <code>bcryptjs</code> , <code>jsonwebtoken</code>	REST API, authentication, authorization (592 lines)
Frontend	HTML5, CSS3, JavaScript (ES modules), Chart.js 4	Single-page app for all four roles, no build step (1,379 lines)
Hosting	Render (web service), Supabase (database)	Live deployment with automatic redeploy on every Git push
Testing	Playwright (Chromium), curl, smoke-test script	End-to-end UI walk-through, API rule tests, post-deploy checks

2.4 Database Design

2.4.1 Relational Schema

Figure 2 shows the eleven relations. Arrows point from a foreign key to the referenced table.

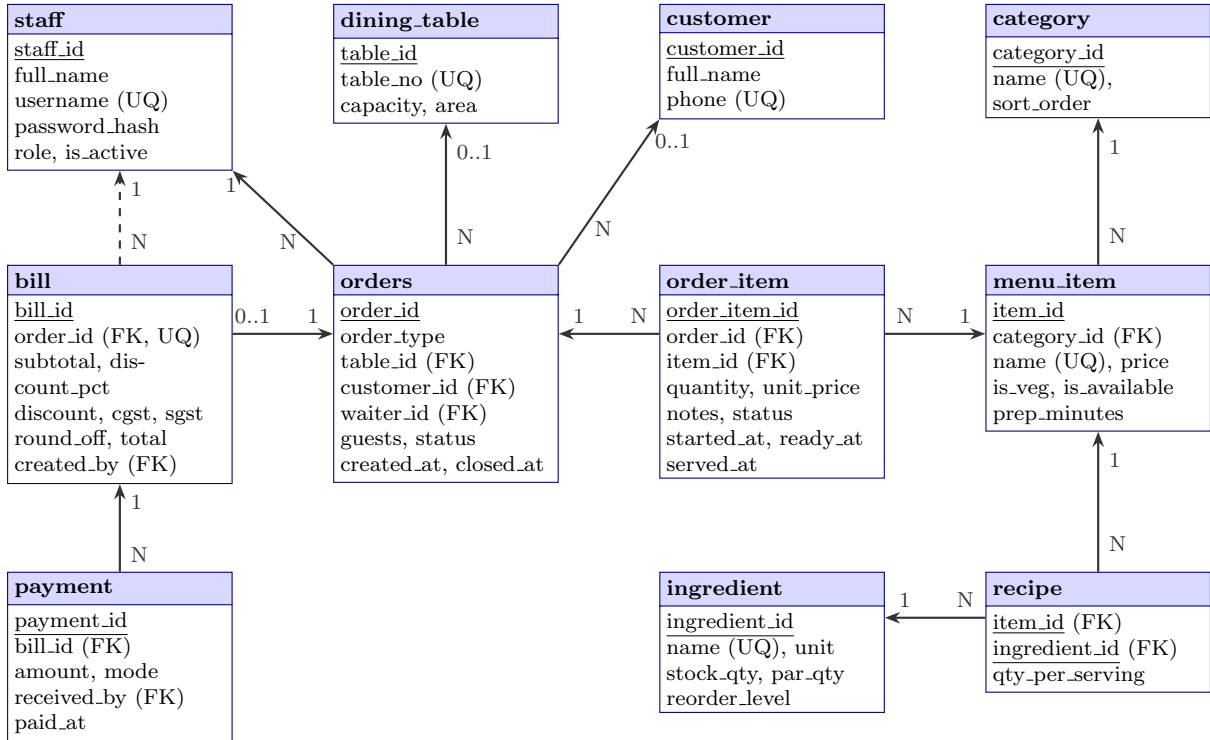


Figure 2: Relational schema of the RMS (PK underlined, FK = foreign key, UQ = unique)

2.4.2 Normalization and Design Decisions

Every relation is in BCNF. Each non-key attribute depends only on the key of its own table, and the many-to-many link between dishes and ingredients is resolved by `recipe`. Four decisions deserve explanation:

1. **Price snapshot.** `order_item.unit_price` looks like a copy of `menu_item.price`, but it is not redundant. It records the price *at the moment of sale*, so a later price change never alters an old bill.

2. **Table status is derived, not stored.** The view `v_table_status` computes `FREE`, `OCCUPIED`, `FOOD_READY` or `BILLING` from the table's open order. A stored status column could fall out of step with the orders; a derived one cannot.
3. **Order status is controlled redundancy.** `orders.status` could be computed from the dish statuses, but it is stored for fast filtering. It is kept correct by a trigger, so no program can set it to a contradictory value.
4. **One open order per table** is enforced by a partial unique index:

Listing 1: Key constraints from `01_schema.sql`

```

1 CREATE TABLE orders (
2     order_id      SERIAL          PRIMARY KEY,
3     order_type    VARCHAR(8)      NOT NULL CHECK (order_type IN ('DINE_IN','TAKEAWAY')),
4     table_id      INT             REFERENCES dining_table,
5     customer_id   INT             REFERENCES customer,
6     waiter_id     INT             NOT NULL REFERENCES staff,
7     guests        SMALLINT        CHECK (guests BETWEEN 1 AND 20),
8     status        VARCHAR(10)     NOT NULL DEFAULT 'PLACED'
9     CHECK (status IN ('PLACED','PREPARING','READY','SERVED','BILLED','PAID','CANCELLED')),
10    created_at    TIMESTAMPTZ     NOT NULL DEFAULT now(),
11    closed_at     TIMESTAMPTZ,
12    CHECK ((order_type = 'DINE_IN') = (table_id IS NOT NULL))    -- dine-in <=> has a table
13 );
14 -- A table can have only ONE open order at a time
15 CREATE UNIQUE INDEX uq_open_order_per_table ON orders (table_id)
16     WHERE status NOT IN ('PAID','CANCELLED');
17
18 CREATE TABLE ingredient (
19     ingredient_id SERIAL          PRIMARY KEY,
20     name          VARCHAR(40)     NOT NULL UNIQUE,
21     unit          VARCHAR(4)      NOT NULL CHECK (unit IN ('kg','l','pcs')),
22     stock_qty     NUMERIC(10,3)   NOT NULL CHECK (stock_qty >= 0),    -- never negative
23     reorder_level NUMERIC(10,3)   NOT NULL DEFAULT 0,
24     par_qty       NUMERIC(10,3)   NOT NULL DEFAULT 0,
25     cost_per_unit NUMERIC(8,2)    NOT NULL DEFAULT 0
26 );

```

2.5 Business Rules Inside the Database

2.5.1 Order Life-Cycle

Each dish of an order moves through the kitchen independently, and the order status follows from its dishes (Figure 3).

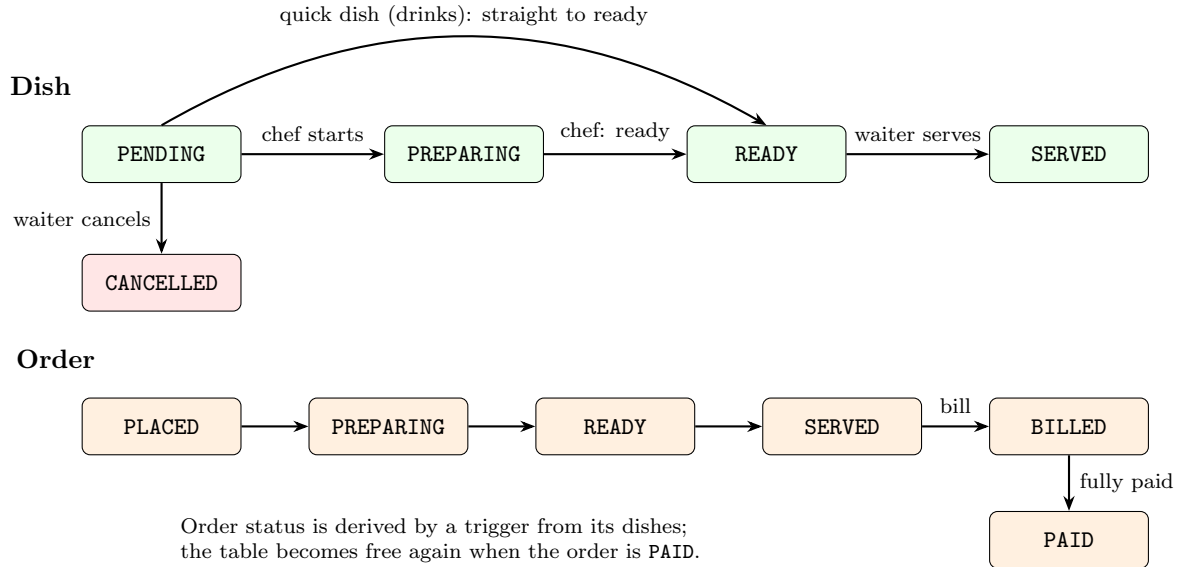


Figure 3: State machines enforced by the database

2.5.2 Triggers

Two row-level triggers on `order_item` carry most of the logic. The `BEFORE` trigger rejects illegal moves and records timestamps. The `AFTER` trigger deducts (or returns) ingredient stock and recalculates the order status.

Listing 2: State-machine trigger (from `02_logic.sql`)

```

1 CREATE FUNCTION trg_order_item_rules() RETURNS TRIGGER
2 LANGUAGE plpgsql AS $$
3 DECLARE
4     v_order_status VARCHAR;
5 BEGIN
6     IF is_seeding() THEN RETURN NEW; END IF;           -- demo-history loader only
7
8     SELECT status INTO v_order_status FROM orders WHERE order_id = NEW.order_id;
9     IF v_order_status IN ('BILLED','PAID','CANCELLED') THEN
10         RAISE EXCEPTION 'Order % is % and can no longer be changed', NEW.order_id, v_order_status;
11     END IF;
12     IF TG_OP = 'INSERT' THEN
13         NEW.status := 'PENDING';
14         RETURN NEW;
15     END IF;
16     IF NEW.status = OLD.status THEN RETURN NEW; END IF;
17     IF NOT ((OLD.status = 'PENDING' AND NEW.status IN ('PREPARING','READY','CANCELLED'))
18             OR (OLD.status = 'PREPARING' AND NEW.status = 'READY')
19             OR (OLD.status = 'READY' AND NEW.status = 'SERVED')) THEN
20         RAISE EXCEPTION 'Item cannot move from % to %', OLD.status, NEW.status;
21     END IF;
22     IF NEW.status IN ('PREPARING','READY') AND NEW.started_at IS NULL THEN
23         NEW.started_at := now();
24     END IF;
25     IF NEW.status = 'READY' THEN NEW.ready_at := now(); END IF;
26     IF NEW.status = 'SERVED' THEN NEW.served_at := now(); END IF;
27     RETURN NEW;
28 END $$;
29
30 CREATE TRIGGER order_item_rules
31 BEFORE INSERT OR UPDATE OF status ON order_item
32 FOR EACH ROW EXECUTE FUNCTION trg_order_item_rules();

```

Listing 3: Stock-deduction trigger (abridged)

```

1 CREATE FUNCTION trg_order_item_effects() RETURNS TRIGGER
2 LANGUAGE plpgsql AS $$

```

```

3 DECLARE
4     v_short RECORD;
5 BEGIN
6     IF is_seeding() THEN RETURN NULL; END IF;
7     IF TG_OP = 'INSERT' THEN
8         -- lock the ingredient rows in a fixed order (prevents deadlocks)
9         PERFORM 1 FROM ingredient
10        WHERE ingredient_id IN (SELECT ingredient_id FROM recipe WHERE item_id = NEW.item_id)
11        ORDER BY ingredient_id FOR UPDATE;
12        SELECT m.name AS dish, i.name AS ingredient, i.unit, i.stock_qty,
13              r.qty_per_serving * NEW.quantity AS needed INTO v_short
14        FROM recipe r JOIN ingredient i ON i.ingredient_id = r.ingredient_id
15              JOIN menu_item m ON m.item_id = r.item_id
16        WHERE r.item_id = NEW.item_id AND i.stock_qty < r.qty_per_serving * NEW.quantity
17        LIMIT 1;
18        IF FOUND THEN
19            RAISE EXCEPTION 'Not enough % for % x % (need %, only % left)',
20                          v_short.ingredient, NEW.quantity, v_short.dish,
21                          v_short.needed, v_short.unit, v_short.stock_qty, v_short.unit;
22        END IF;
23        UPDATE ingredient i SET stock_qty = i.stock_qty - r.qty_per_serving * NEW.quantity
24        FROM recipe r WHERE r.item_id = NEW.item_id AND r.ingredient_id = i.ingredient_id;
25        ELSIF NEW.status = 'CANCELLED' AND OLD.status = 'PENDING' THEN
26            UPDATE ingredient i SET stock_qty = i.stock_qty + r.qty_per_serving * NEW.quantity
27            FROM recipe r WHERE r.item_id = NEW.item_id AND r.ingredient_id = i.ingredient_id;
28        END IF;
29        PERFORM fn_refresh_order_status(NEW.order_id);    -- derive the order status
30        RETURN NULL;
31 END $$;

```

2.5.3 Stored Functions

The API never writes order rows directly. It calls one function per business action, and each call runs as a single atomic transaction:

Table 2: Business functions

Function	What it guarantees
fn_place_order	Valid table, table not already occupied, guests $\leq$ capacity, at least one dish; serialised per table with an advisory lock; creates or reuses the takeaway customer by phone.
fn_add_items	Order still open; each dish exists and is available; quantity 1–50; price snapshot taken.
fn_cancel_order	Cancels only if the kitchen has not started any dish; stock is returned by the trigger.
fn_generate_bill	Only when every dish is served; discount 0–50%; GST and rounding computed in one place.
fn_record_payment	No over-payment; supports split payments (cash + UPI); closes the order and frees the table at zero balance.

Listing 4: Placing an order (abridged from fn_place_order)

```

1 IF p_type = 'DINE_IN' THEN
2     -- serialise concurrent orders for the same table
3     PERFORM pg_advisory_xact_lock(hashtext('rms.table'), COALESCE(p_table, 0));
4     SELECT capacity INTO v_capacity FROM dining_table WHERE table_id = p_table;
5     IF NOT FOUND THEN RAISE EXCEPTION 'Choose a valid table'; END IF;
6     IF EXISTS (SELECT 1 FROM orders WHERE table_id = p_table
7              AND status NOT IN ('PAID', 'CANCELLED')) THEN
8         RAISE EXCEPTION 'This table already has an open order';
9     END IF;
10    IF p_guests IS NULL OR p_guests < 1 OR p_guests > v_capacity THEN
11        RAISE EXCEPTION 'This table seats 1 to % guests', v_capacity;

```

```

12     END IF;
13 END IF;
14 INSERT INTO orders (order_type, table_id, customer_id, waiter_id, guests)
15 VALUES (p_type, p_table, v_customer, p_waiter, p_guests) RETURNING order_id INTO v_order;
16 PERFORM fn_add_items(v_order, p_items);    -- inserts dishes; triggers deduct stock
17 RETURN v_order;

```

2.5.4 Mathematical Model

Stock deduction. When n plates of dish m are ordered, every ingredient i in its recipe is reduced by

$$s_i \leftarrow s_i - n \cdot q_{m,i}, \quad \text{subject to } s_i \geq 0,$$

where $q_{m,i}$ is the quantity per serving. The CHECK (`stock_qty >= 0`) constraint makes the condition absolute, even for simultaneous orders.

Plates still possible. The view `v_menu_stock` tells the waiter how many more plates of a dish the current stock allows:

$$\text{servings}(m) = \min_{i \in \text{recipe}(m)} \left\lfloor \frac{s_i}{q_{m,i}} \right\rfloor$$

GST bill. Restaurant service in India is taxed at 5% GST (CGST 2.5% + SGST 2.5%). With subtotal $S = \sum_j q_j p_j$ and discount rate d :

$$D = \text{round}_2\left(S \cdot \frac{d}{100}\right), \quad T = S - D, \\ \text{CGST} = \text{SGST} = \text{round}_2(0.025 T), \quad \text{Total} = \text{round}_0(T + \text{CGST} + \text{SGST}).$$

Worked example (live test, order of 2 Chicken Biryani, 1 Butter Chicken, 3 Butter Naan, 1 Irani Chai): $S = 640 + 340 + 180 + 40 = 1200$; with $d = 10$, $D = 120$, $T = 1080$, $\text{CGST} = \text{SGST} = 27.00$, $\text{Total} = \text{Rs. } 1,134$. The receipt in Figure 7 shows exactly these figures.

2.6 Analytical Queries for the Dashboard

The manager’s dashboard is built from SQL aggregate and window queries. Two examples:

Listing 5: Top dishes (window function) and kitchen speed

```

1  -- Top dishes in the selected period
2  SELECT RANK() OVER (ORDER BY SUM(oi.quantity) DESC) AS rnk, m.name,
3         SUM(oi.quantity) AS qty, SUM(oi.quantity * oi.unit_price) AS revenue
4  FROM order_item oi
5  JOIN orders o ON o.order_id = oi.order_id
6  JOIN menu_item m ON m.item_id = oi.item_id
7  WHERE o.status = 'PAID' AND o.created_at >= $1 AND oi.status <> 'CANCELLED'
8  GROUP BY m.item_id, m.name
9  ORDER BY rnk LIMIT 8;
10
11 -- Average preparation time and order-to-table time per dish
12 SELECT m.name, m.prep_minutes AS target,
13        ROUND(AVG(EXTRACT(EPOCH FROM (oi.ready_at - oi.started_at)) / 60)::numeric, 1) AS avg_prep,
14        ROUND(AVG(EXTRACT(EPOCH FROM (oi.served_at - oi.created_at)) / 60)::numeric, 1) AS avg_to_table
15 FROM order_item oi
16 JOIN orders o ON o.order_id = oi.order_id
17 JOIN menu_item m ON m.item_id = oi.item_id
18 WHERE o.created_at >= $1 AND oi.served_at IS NOT NULL
19 GROUP BY m.item_id, m.name, m.prep_minutes
20 ORDER BY avg_to_table DESC LIMIT 6;

```

Results on the live database (30 days of generated demo history, 1,026 orders):

Table 3: Live query results (last 30 days)

#	Dish	Plates	Rs.	Dish	Target	Prep	To table
1	Butter Naan	686	41,160	Mutton Biryani	30	30.6	36.0
2	Hyd. Chicken Biryani	413	1,32,160	Chicken Biryani	25	25.4	30.9
3	Tandoori Roti	296	10,360	Butter Chicken	20	20.3	25.8
4	Irani Chai	292	11,680	Egg Biryani	20	20.1	25.4
5	Butter Chicken	226	76,840				

(minutes)

Over the 30 days the system recorded 978 paid orders, revenue of Rs. 7,85,748 and an average bill of Rs. 803. Payments were 54% UPI, 25% cash and 20% card, and the busiest hours were 8–9 PM (243 orders) and 1–2 PM (231 orders).

2.7 Application Layer

2.7.1 REST API

Table 4: Main API endpoints (all except login require a JWT)

Method	Path	Roles	Database object
POST	/api/auth/login	public	staff (bcrypt check)
GET	/api/tables	all	view v_table_status
GET	/api/menu	all	menu_item + view v_menu_stock
POST	/api/orders	Waiter, Manager	fn_place_order
POST	/api/orders/:id/items	Waiter, Manager	fn_add_items
PATCH	/api/order-items/:id	Chef (cook), Waiter (serve)	UPDATE order_item + triggers
GET	/api/kitchen	Chef, Waiter, Manager	view v_kitchen_queue
POST	/api/orders/:id/bill	Cashier, Manager	fn_generate_bill
POST	/api/bills/:id/payments	Cashier, Manager	fn_record_payment
GET/POST	/api/inventory	Chef (view), Manager	ingredient
GET	/api/reports/summary	Manager	10 aggregate queries

Listing 6: A route: the API only validates input and calls the database function

```

1 router.post('/orders', allow('WAITER'), wrap(async (req, res) => {
2   const { order_type, table_id, guests, customer_name, phone, items } = req.body;
3   const { rows } = await db.query(
4     'SELECT fn_place_order($1, $2, $3, $4, $5, $6, $7::jsonb) AS order_id',
5     [req.user.staff_id, order_type, int(table_id), int(guests), customer_name || null,
6       phone || null, JSON.stringify(items || [])]
7   );
8   res.status(201).json(rows[0]);
9 });

```

Errors raised by PL/pgSQL (`RAISE EXCEPTION`) reach the browser as clear messages, for example “*Not enough Mutton for 20 x Mutton Dum Biryani (need 5.000 kg, only 4.000 kg left)*”.

2.7.2 Security Measures

- Passwords are stored only as **bcrypt** hashes; a login returns a signed **JWT** valid for 12 hours, and repeated failed logins are throttled.
- Every route checks the user’s **role**; the database itself is reached through the role **rms_app**, which has no **DELETE** or **TRUNCATE** rights, cannot change staff accounts, and cannot edit bills or payments once created.

- All SQL uses **bound parameters** (\$1, \$2, ...), so SQL injection is not possible; the database connection uses TLS.
- The server sends a strict **Content-Security-Policy** and other security headers, and all text shown in the browser is HTML-escaped to prevent cross-site scripting.

2.8 User Interface

The frontend is a single-page application that adapts its menu to the logged-in role. Tables, kitchen and billing screens refresh automatically every few seconds, so all devices show the same live state.

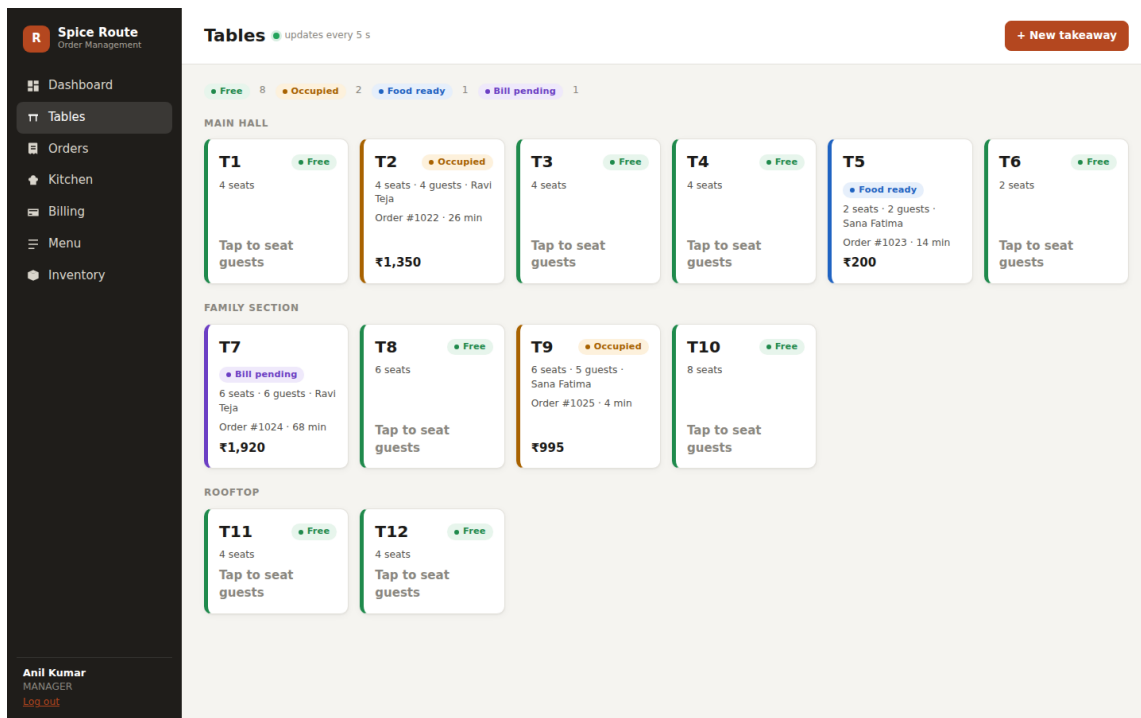


Figure 4: Waiter's floor plan: each table's state is derived live from its open order

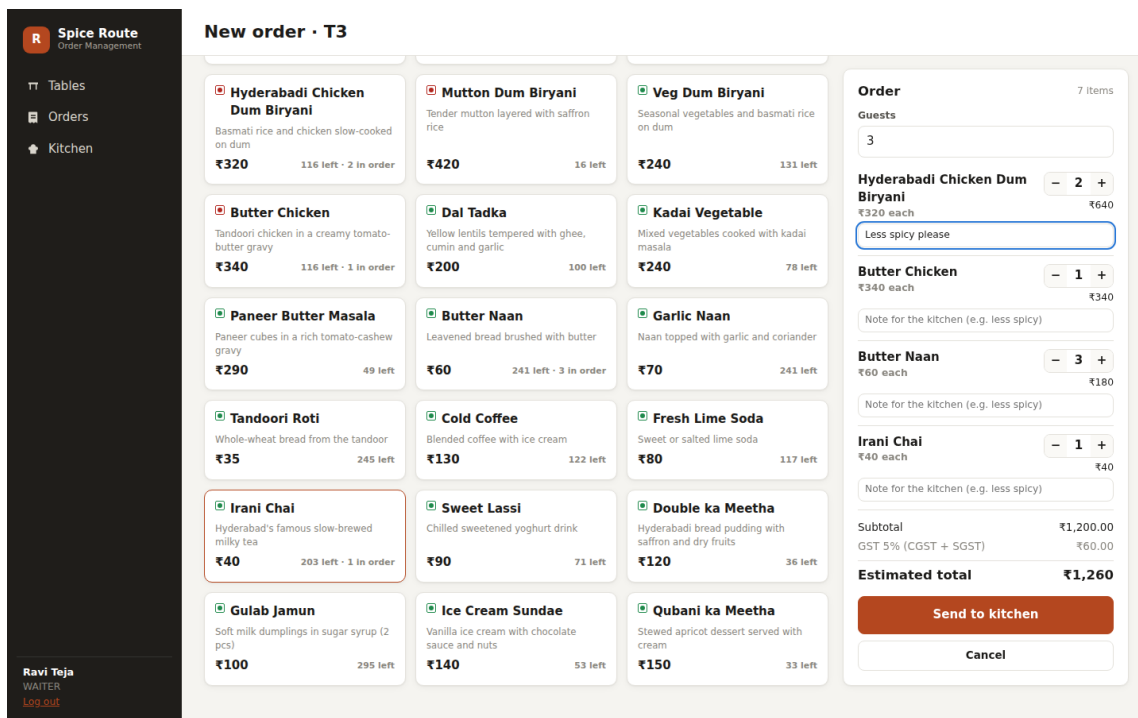


Figure 5: Taking an order: menu with live “plates left” from stock, cart with kitchen notes and GST estimate

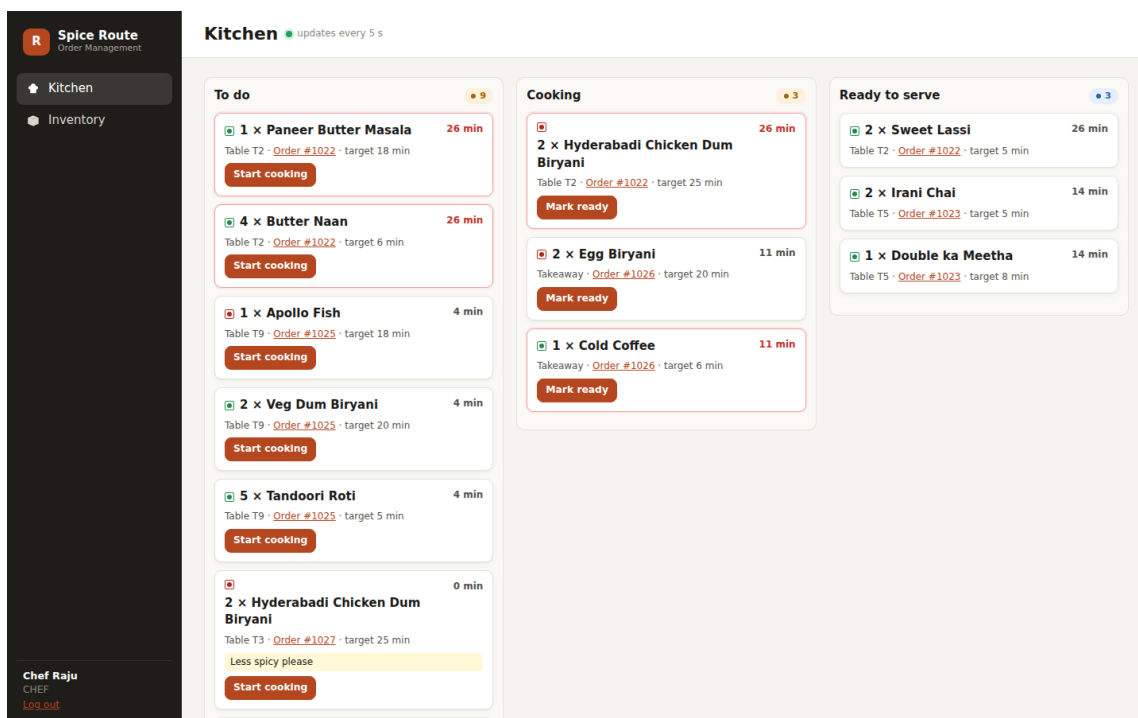


Figure 6: Kitchen display system: dishes waiting longer than their target time are highlighted in red

R

Spice Route

Order Management

Orders

Billing

Lakshmi Devi

CASHIER

Log out

Billing

updates every 10 s

Waiting at the counter

T7 #1024

Ravi Teja - 9:47 am

₹2,016

Billed

T3 #1027

Ravi Teja - 10:55 am

₹1,134

Billed

SPICE ROUTE

Hyderabad

Tax invoice (demo)

Bill #1023

5 Oct, 10:55 am

Table T3 - 3 guests

Order #1027

2 x Hyderabad Chicken Dum Biryani

₹640.00

1 x Butter Chicken

₹340.00

3 x Butter Naan

₹180.00

1 x Irani Chai

₹40.00

Subtotal

₹1,200.00

Discount 10%

-₹120.00

CGST 2.5%

₹27.00

SGST 2.5%

₹27.00

Round off

0.00

TOTAL

₹1,134.00

Thank you! Visit again.

Print receipt

Payment

Billed

Bill total

₹1,134.00

Paid so far

₹0.00

Balance

₹1,134.00

Mode

UPI

Cash

Card

Amount

1134

Cash received

2000

Return change: ₹866.00

Record payment

Figure 7: Billing counter: GST receipt (10% discount applied) and cash payment with change calculation

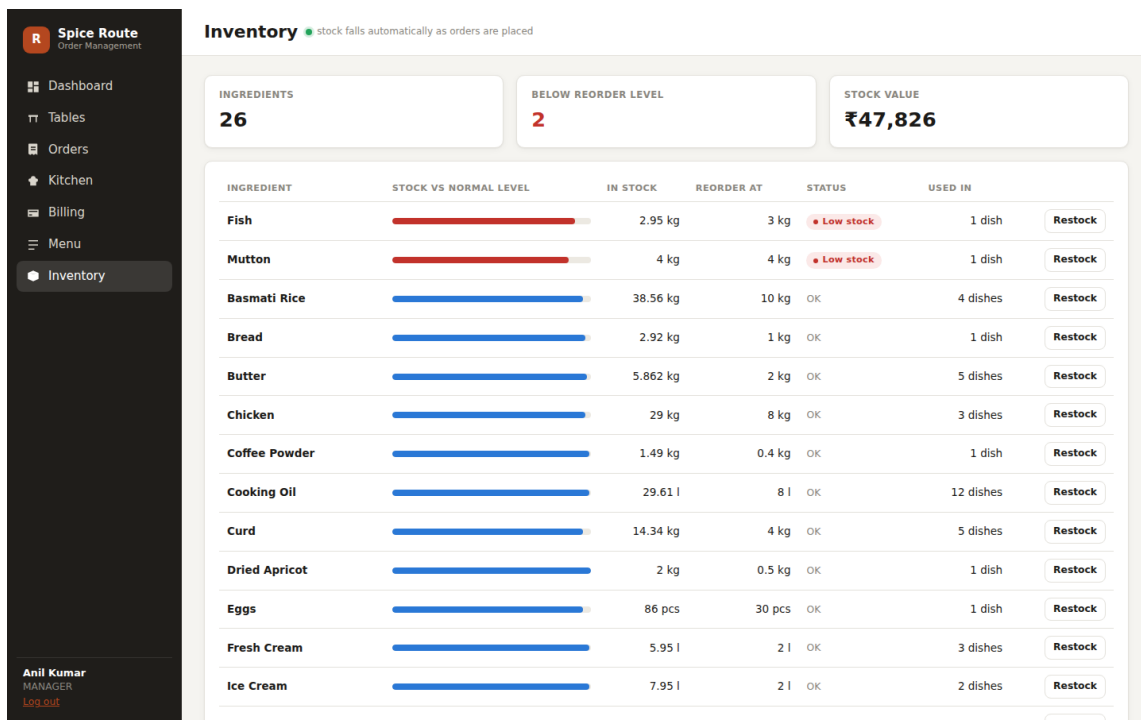


Figure 9: Inventory: stock is reduced automatically by the trigger; items below reorder level are flagged

Spice Route
Restaurant Order Management System

Username
Password

Log in

Demo accounts (click to sign in):

Anil Kumar Manager - manager / manager123	Ravi Teja Waiter - waiter / waiter123
Chef Raju Kitchen - chef / chef123	Lakshmi Devi Cashier - cashier / cashier123

(a) Login with demo accounts

Spice Route
Order Management

Tables Orders Kitchen

Order #1027 Placed + Add dishes Cancel order

TABLE	PLACED	RUNNING TOTAL
T3 3 guests	10:55 am 0 min ago · by Ravi Teja	₹1,200 before GST

DISH	QTY	PRICE	AMOUNT	STATUS	TIMELINE
Hyderabadi Chicken Dum Biryani Note: Less spicy please	2	₹320	₹640	To do	ordered 10:55 am Cancel
Butter Chicken	1	₹340	₹340	To do	ordered 10:55 am Cancel
Butter Naan	3	₹60	₹180	To do	ordered 10:55 am Cancel
Irani Chai	1	₹40	₹40	To do	ordered 10:55 am Cancel

(b) Order detail with per-dish status and timeline

Spice Route
Order Management

Tables Orders Kitchen

Ravi Teja
WAITER
[Log out](#)

Tables + New takeaway

updates every 5 s

Free 8 **Occupied** 2 **Food ready** 1
Bill pending 1

MAIN HALL

T1 Free
4 seats
Tap to seat guests

T2 Occupied
4 seats · 4 guests · Ravi Teja
Order #1022 · 26 min
₹1,350

T3 Free
4 seats
Tap to seat guests

T4 Free
4 seats
Tap to seat guests

T5 Food ready
2 seats · 2 guests · Sana Fatima
Order #1023 · 14 min
₹200

T6 Free
2 seats
Tap to seat guests

FAMILY SECTION

T7 Bill pending
6 seats · 6 guests · Ravi Teja
Order #1024 · 68 min
₹1,920

T8 Free
6 seats
Tap to seat guests

T9 Occupied
6 seats · 5 guests · Sana Fatima
Order #1025 · 4 min
₹995

T10 Free
8 seats
Tap to seat guests

ROOFTOP

T11 Free
4 seats
Tap to seat guests

T12 Free
4 seats
Tap to seat guests

(c) Phone layout

Figure 10: Login, order detail and responsive mobile view

2.9 Deployment

Table 5: Live deployment

Item	Details
Public URL	https://spice-route-rms.onrender.com
Web service	Render, free plan, Singapore region, Node.js 22; rebuilt automatically on every push to the Git branch
Database	Supabase PostgreSQL 17.11, Mumbai region (ap-south-1); schema rms created through four SQL migrations
Connection	Supavisor connection pooler (session mode, TLS), user rms_app
Configuration	Environment variables DATABASE_URL , JWT_SECRET , NODE_VERSION ; no secrets are stored in Git
Note	On the free plan the server sleeps after 15 minutes without visitors; the first request then takes up to about a minute to wake it.

2.10 Testing and Validation

The system was tested at three levels: business-rule tests against the API, an automated end-to-end walk-through in a real browser (Playwright with Chromium), and a smoke test that the deployed server runs against itself after start-up.

Table 6: Test cases and observed results

#	Test	Observed result	Pass
1	Login with a wrong password	401 <i>Invalid username or password</i>	✓
2	API call without logging in	401 <i>Please log in</i>	✓
3	Waiter places a dine-in order (3 guests, 3 dishes)	Order created, dishes PENDING , stock reduced	✓
4	Second order on an occupied table	<i>This table already has an open order</i>	✓
5	Order with no dishes	<i>Add at least one item</i>	✓
6	Chef tries to place an order	403 <i>This action needs one of: MANAGER, WAITER</i>	✓
7	20 Mutton Biryani with 4 kg mutton in stock	<i>Not enough Mutton ... (need 5.000 kg, only 4.000 kg left)</i>	✓
8	Waiter marks a dish PREPARING	403 <i>Only CHEF or MANAGER can ...</i>	✓
9	Dish moved PREPARING → SERVED (skipping READY)	<i>Item cannot move from PREPARING to SERVED</i>	✓
10	All dishes marked ready	Order status became READY automatically (trigger)	✓
11	Bill requested before all dishes served	<i>A bill can be generated only after every item is served</i>	✓
12	Bill for Rs. 940 with 10% discount	Total Rs. 888 (CGST = SGST = 21.15, round-off −0.30)	✓
13	Over-payment of Rs. 900 on Rs. 888	<i>Amount must be between 0.01 and 888.00</i>	✓
14	Split payment Rs. 500 cash + Rs. 388 UPI	Balance 0, order PAID , table FREE	✓
15	Payment on a settled bill	<i>This bill is already settled</i>	✓
16	Ordering a dish hidden from the menu	<i>Ice Cream Sundae is not available right now</i>	✓
17	Cashier opens the manager’s report	403 Forbidden	✓
18	Browser walk-through: waiter → chef → waiter → cashier	Order taken, cooked, served, billed (Rs. 1,134) and paid; 0 browser errors	✓
19	Post-deploy smoke test on the live server	18/18 checks passed (all roles, all main screens)	✓

2.11 Proposed Solution Strategy

Table 7: Summary of the solution

Strategy	Implementation Details
Data Model	11 normalized tables; price snapshot per order line; derived table status; partial unique index for one open order per table.
Rules in the DB	State-machine and stock triggers; atomic PL/pgSQL functions for ordering, billing and payment.
Concurrency	Advisory lock per table, ordered row locks on ingredients, <code>CHECK (stock_qty >= 0)</code> as the final guarantee.
Application	Express REST API, JWT + bcrypt login, role checks, parameterized SQL, least-privilege DB role.
Interface	Role-aware single-page app: floor plan, order taking, kitchen display, billing, menu, inventory, dashboard.
Deployment	Render web service + Supabase PostgreSQL, automatic re-deploy, post-deploy smoke test.

3 Sustainability Integration

3.1 Reducing Food Waste and Paper

Every dish has a recipe, so the database always knows how much of each ingredient remains. The kitchen therefore sees shortages *before* a dish is promised to a customer. The dashboard’s top-dish and hourly-demand data allow the manager to prepare and purchase in line with actual demand, the core lever for reducing food waste (**SDG 12**, Target 12.3). Digital kitchen tickets and on-screen bills also replace thousands of paper KOT slips each month.

3.2 Impact on Society and Environment

- **Social and economic:** Accurate bills with correct GST build customer trust and protect the restaurant’s revenue. Clear role-based screens reduce staff stress and errors, supporting decent work (**SDG 8**).
- **Technological inclusion:** The system runs in any browser on existing phones and PCs and is hosted on free-tier cloud services, so a small restaurant can adopt it at almost no cost (**SDG 9**).
- **Environmental:** The lean design (no build step, about 200 KB of JavaScript, short polling intervals only on live screens) keeps network and server energy use low.

4 Conclusion and Future Directions

4.1 Solution Summary

The project delivered a working, publicly deployed Restaurant Order Management System. A normalized PostgreSQL schema of eleven tables, with triggers and stored functions, guarantees correct order and dish states, automatic stock deduction, one open order per table and exact

GST billing, regardless of which program changes the data. A Node.js/Express API with bcrypt, JWT and role checks connects a responsive single-page frontend for waiters, the kitchen, cashiers and the manager. All nineteen tests passed, including a browser walk-through of a complete order and a post-deployment smoke test on the live server.

4.2 Future Scope

Future versions can add **QR-code table ordering** by customers, **UPI payment-gateway** integration, and **WebSocket** push updates in place of polling. Other extensions are **thermal-printer** support for kitchen tickets and receipts, **multi-branch** management, a **loyalty programme** using the customer table, and **machine-learning demand forecasting** for automatic purchase suggestions.

Bibliography

- [1] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th Edition, McGraw-Hill Education, 2019.
- [2] R. Elmasri and S. B. Navathe, *Fundamentals of Database Systems*, 7th Edition, Pearson Education, 2016.
- [3] The PostgreSQL Global Development Group, *PostgreSQL 17 Documentation: Triggers, PL/pgSQL, Explicit Locking and Partial Indexes*, <https://www.postgresql.org/docs/17/>.
- [4] N. Provos and D. Mazières, “A Future-Adaptable Password Scheme,” *Proc. USENIX Annual Technical Conference*, 1999.
- [5] M. Jones, J. Bradley and N. Sakimura, “JSON Web Token (JWT),” RFC 7519, IETF, 2015.
- [6] R. T. Fielding, *Architectural Styles and the Design of Network-based Software Architectures*, Ph.D. Dissertation, University of California, Irvine, 2000.
- [7] OWASP Foundation, *OWASP Top 10:2021 and SQL Injection Prevention Cheat Sheet*, <https://owasp.org>.
- [8] Central Board of Indirect Taxes and Customs, Notification No. 46/2017-Central Tax (Rate): GST on restaurant services, 2017.
- [9] United Nations, *Transforming Our World: The 2030 Agenda for Sustainable Development*, A/RES/70/1, 2015.